EXPERIMENT-3

SACHIN SINGH

24UADS1045

Objective :-

WAP to implement a three-layer neural network using Tensor flow library (only, no keras) to classify MNIST handwritten digits dataset. Demonstrate the implementation of feed-forward and back-propagation approaches.

Description Of the Model :-

Title:

Implementation of a Three-Layer Neural Network using TensorFlow (without Keras) for MNIST Digit Classification

1. Overview

The model is a three-layer fully connected neural network implemented using the TensorFlow core library (without using Keras high-level APIs).

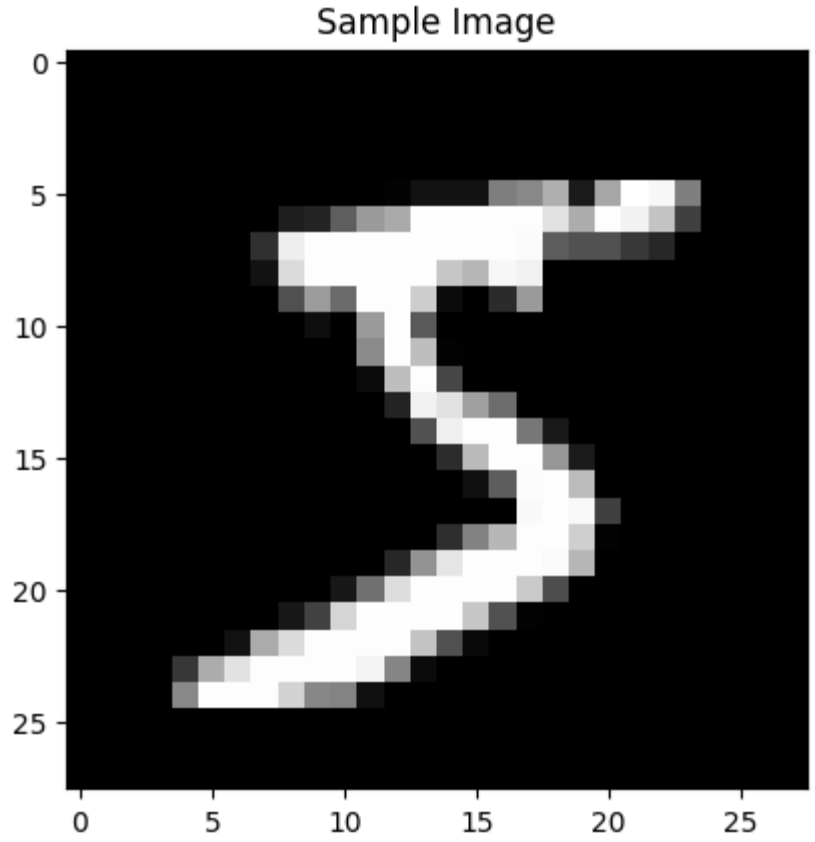
It is designed to classify handwritten digit images from the MNIST dataset into 10 classes (digits 0–9).

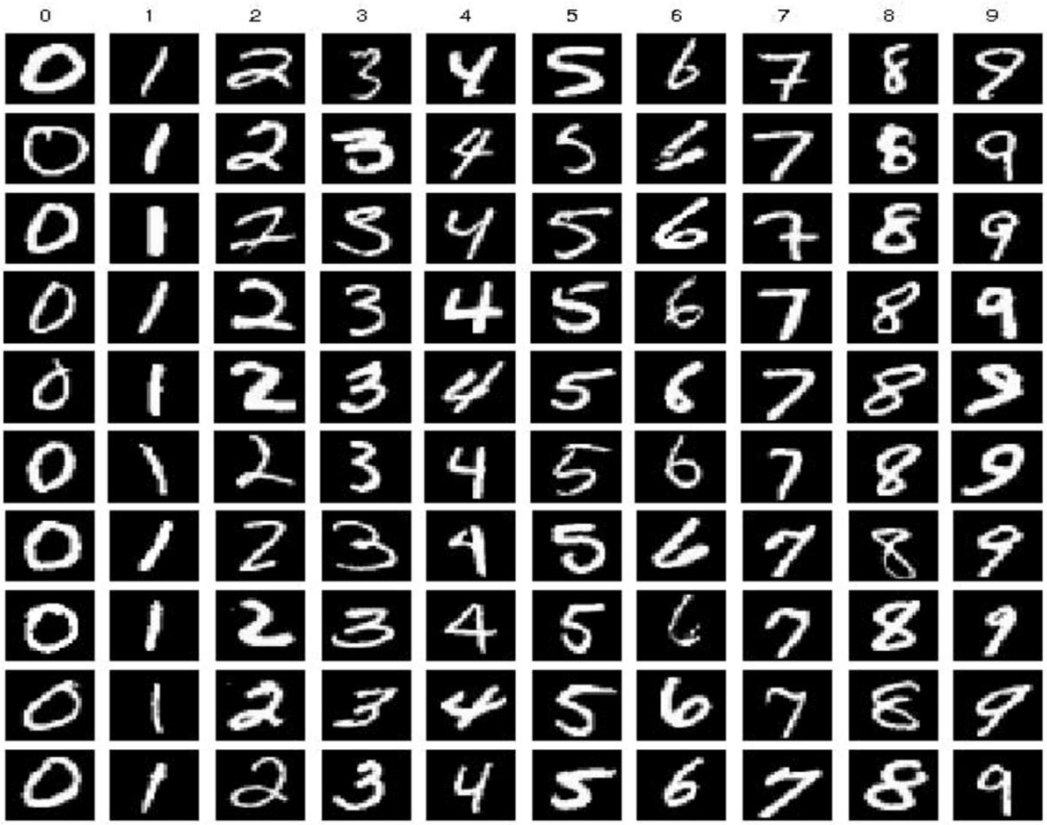
The network uses:

- Mini-batch Gradient Descent
- Forward Propagation
- Backpropagation using automatic differentiation (tf.GradientTape)

2. Dataset Used

MNIST Handwritten Digits Dataset


Sample Image
0
5
10
15
20
25
0
5
10
15
20
25


0
1
2
3
4
5
6
7
8
9

- Total training samples: 60,000
- Test samples: 10,000
- Image size: 28 × 28 pixels
- Input features after flattening: 784
- Output classes: 10

Each image is normalized between 0 and 1.

3. Network Architecture

The neural network consists of:

Layer	Type	Neurons	Activation
Input Layer	Fully Connected	784	—
Hidden Layer	Fully Connected	128	ReLU
Output Layer	Fully Connected	10	Softmax

Architecture Flow :-

Input (784)

↓

Z1 = XW1 + b1

↓

A1 = ReLU(Z1)

↓

Z2 = A1W2 + b2

↓

A2 = Softmax(Z2)

---

4. Mathematical Representation

Forward Propagation

1. Hidden Layer:
   Z1=XW1+b1

   A1=ReLU(Z1)

2. Output Layer:
   $Z2=A1W2+b2$

   $A2=Softmax(Z2)$

Loss Function

Categorical Cross-Entropy Loss:

$Loss=-\sum y\log(y^)$

Backpropagation :-

Gradients are computed using TensorFlow's automatic differentiation:

$$W=W-\eta(\partial L/\partial W)$$

Where:

- η = Learning Rate
- L = Loss Function

5. Training Method

The model is trained using:

Mini-Batch Gradient Descent

Instead of updating weights using the entire dataset, the data is divided into smaller batches (e.g., 128 samples).

Advantages:

- Faster convergence
- More stable updates
- Reduced memory usage

6. Weight Initialization

Weights are initialized using random normal distribution:

$$W1 \in R^{784x128}$$

$$W2 \in R^{784x10}$$

Bias vectors:

$$b1 \in R^{128}$$

$$b2 \in R^{10}$$

Initial and final values of W1 and W2 are printed to observe weight updates.

7. Model Performance

During training, the following are monitored:

- Loss per epoch
- Accuracy per epoch
- Weight update magnitude
- Gradient norms per layer

Expected performance:

- Initial Loss ≈ 2.3
- Final Loss < 0.3
- Accuracy ≈ 90–97%

8. Why This Model Works

- ReLU avoids vanishing gradient in hidden layer
- Softmax provides probability distribution over 10 classes
- Cross-entropy works well for multi-class classification
- Mini-batch GD improves optimization efficiency

9. Conclusion

The implemented three-layer neural network successfully classifies handwritten digits using forward and backward propagation.

The model demonstrates:

- Effective weight learning
- Decreasing loss over epochs
- Increasing classification accuracy

It validates the working of feed-forward computation and backpropagation in deep learning systems using TensorFlow core operations.

Source Code : -

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt


# Fix random seed

tf.random.set_seed(42)


# Load MNIST dataset

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()


# Normalize and reshape

X_train = X_train.reshape(-1, 784) / 255.0

X_test = X_test.reshape(-1, 784) / 255.0


# One-hot encoding

y_train = tf.one_hot(y_train, depth=10)

y_test = tf.one_hot(y_test, depth=10)


# Convert to tensors

X_train = tf.convert_to_tensor(X_train, dtype=tf.float32)

y_train = tf.convert_to_tensor(y_train, dtype=tf.float32)

X_test = tf.convert_to_tensor(X_test, dtype=tf.float32)

y_test = tf.convert_to_tensor(y_test, dtype=tf.float32)


# Network parameters

input_size = 784

hidden_size = 128

output_size = 10

learning_rate = 0.01

epochs = 10
```

```
# Initialize weights
W1 = tf.Variable(tf.random.normal([input_size, hidden_size], stddev=0.1))
b1 = tf.Variable(tf.zeros([hidden_size]))

W2 = tf.Variable(tf.random.normal([hidden_size, output_size], stddev=0.1))
b2 = tf.Variable(tf.zeros([output_size]))

print("Initial W1 (first 5 values):")
print(W1.numpy().flatten()[:5])

print("\nInitial W2 (first 5 values):")
print(W2.numpy().flatten()[:5])

# Training loop
batch_size = 128
learning_rate = 0.1
epochs = 10

dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train))
dataset = dataset.shuffle(60000).batch(batch_size)

loss_history = []
accuracy_history = []
weight_update_history = []
grad_W1_history = []
grad_W2_history = []
```

```
for epoch in range(epochs):

    epoch_loss = 0
    correct = 0
    total = 0
    total_grad_W1 = 0
    total_grad_W2 = 0

    old_W1 = W1.numpy()

    for x_batch, y_batch in dataset:

        with tf.GradientTape() as tape:

            # -------- Forward Propagation --------
            Z1 = tf.matmul(x_batch, W1) + b1
            A1 = tf.nn.relu(Z1)

            Z2 = tf.matmul(A1, W2) + b2

            loss = tf.reduce_mean(
                tf.nn.softmax_cross_entropy_with_logits(
                    labels=y_batch,
                    logits=Z2
                )
            )
```

```
# -------- Backpropagation --------
        grads = tape.gradient(loss, [W1, b1, W2, b2])

        total_grad_W1 += tf.norm(grads[0]).numpy()
        total_grad_W2 += tf.norm(grads[2]).numpy()

        W1.assign_sub(learning_rate * grads[0])
        b1.assign_sub(learning_rate * grads[1])
        W2.assign_sub(learning_rate * grads[2])
        b2.assign_sub(learning_rate * grads[3])

        epoch_loss += loss.numpy()

        preds = tf.argmax(Z2, axis=1)
        labels = tf.argmax(y_batch, axis=1)

        correct += tf.reduce_sum(tf.cast(preds == labels, tf.float32)).numpy()
        total += y_batch.shape[0]

    acc = correct / total

    # Save history
    loss_history.append(epoch_loss)
    accuracy_history.append(acc)

    weight_update = np.linalg.norm(W1.numpy() - old_W1)
    weight_update_history.append(weight_update)
```

```
 grad_W1_history.append(total_grad_W1)
    grad_W2_history.append(total_grad_W2)

    print(f"Epoch {epoch+1}")
    print(f"Loss: {epoch_loss:.4f}")
    print(f"Accuracy: {acc*100:.2f}%")
    print("-"*40)

print("\nFinal W1 (first 5 values):")
print(W1.numpy().flatten()[:5])

print("\nFinal W2 (first 5 values):")
print(W2.numpy().flatten()[:5])

plt.figure()
plt.plot(loss_history)
plt.title("Loss Decrease over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()

plt.figure()
plt.plot(accuracy_history)
plt.title("Accuracy over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()
```

```
plt.figure()
plt.plot(weight_update_history)
plt.title("Weight Update Magnitude")
plt.xlabel("Epoch")
plt.ylabel("Weight Change (||ΔW1||)")
plt.show()

plt.figure()
plt.plot(grad_W1_history, label="Gradient W1")
plt.plot(grad_W2_history, label="Gradient W2")
plt.legend()
plt.title("Gradient Change per Layer")
plt.xlabel("Epoch")
plt.ylabel("Gradient Norm")
plt.show()
```

Output :-

```
Initial W1 (first 5 values):
[ 0.03274685 -0.08426258  0.03194337 -0.14075519 -0.238806  ]

Initial W2 (first 5 values):
[ 0.00842246 -0.08609038  0.0378123  -0.00051963 -0.0494532 ]

Epoch 1
Loss: 228.4635
Accuracy: 86.35%
----------------------------------------
```

Epoch 2

Loss: 126.8887

Accuracy: 92.15%

Epoch 3

Loss: 103.3088

Accuracy: 93.69%

Epoch 4

Loss: 87.6255

Accuracy: 94.68%

Epoch 5

Loss: 76.3560

Accuracy: 95.38%

Epoch 6

Loss: 67.8188

Accuracy: 95.80%

Epoch 7

Loss: 61.0501

Accuracy: 96.29%

Epoch 8

Loss: 55.7331

Accuracy: 96.65%

Epoch 9

Loss: 51.0193

Accuracy: 96.92%

Epoch 10

Loss: 47.0671

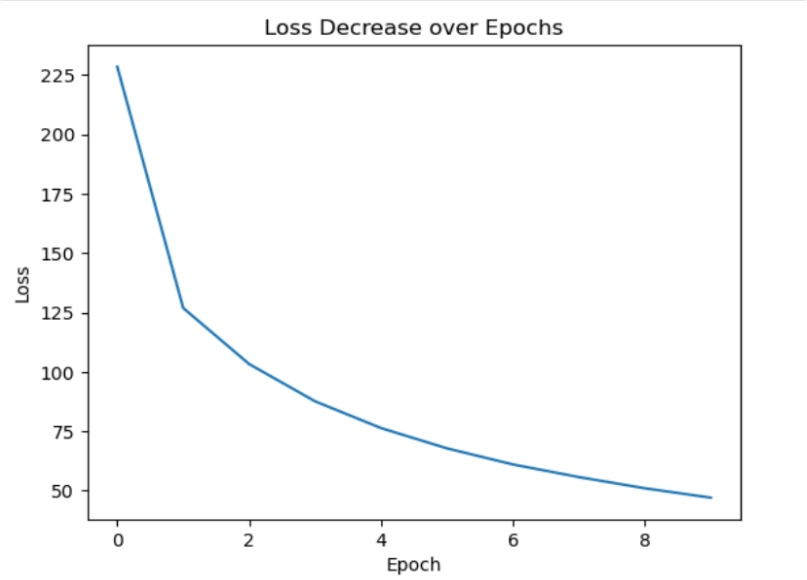
Accuracy: 97.18%

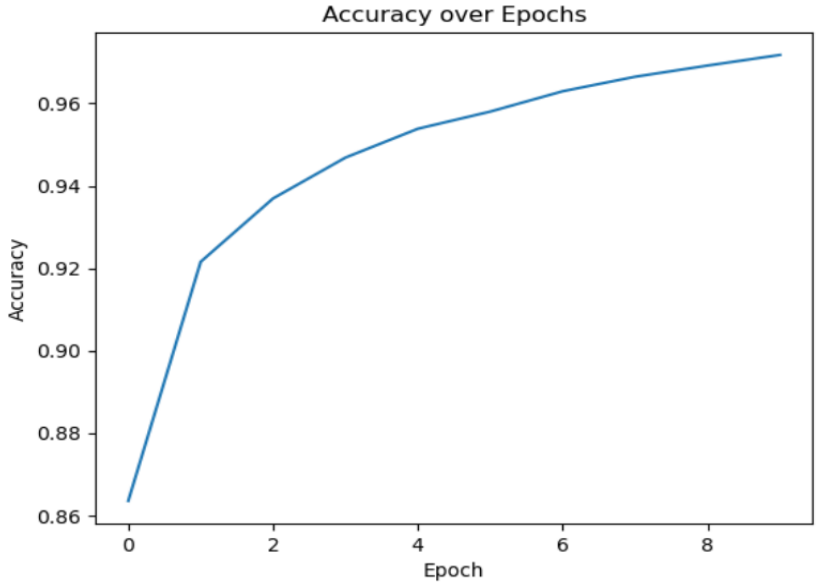
Final W1 (first 5 values):

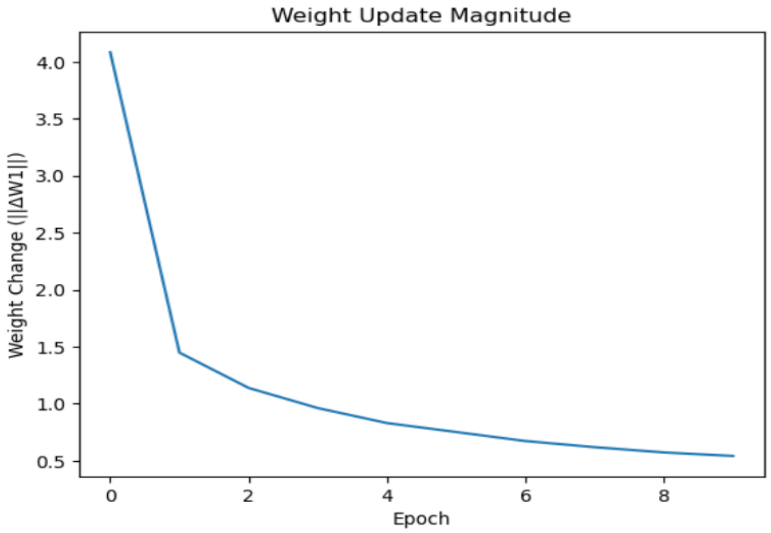
[0.03274685 -0.08426258 0.03194337 -0.14075519 -0.238806]

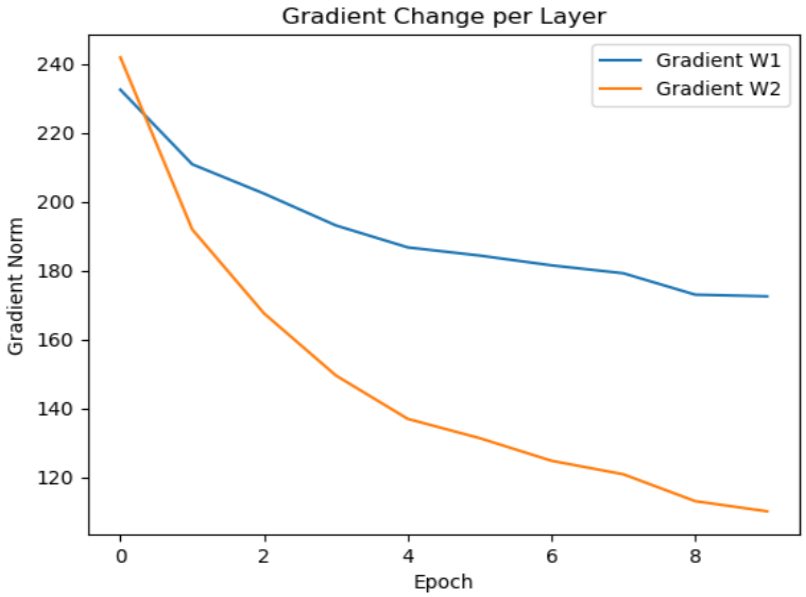
Final W2 (first 5 values):

[-0.18958494 -0.00905381 0.09506208 0.30755168 0.04243352]




Accuracy over Epochs
Accuracy
0.96
0.94
0.92
0.90
0.88
0.86
0
2
4
6
8
Epoch


Weight Update Magnitude
Weight Change (||ΔW1||)
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0
2
4
6
8
Epoch


Gradient Change per Layer
Gradient W1
Gradient W2
Gradient Norm
240
220
200
180
160
140
120
0
2
4
6
8
Epoch

Description of Code :-

- 1. Importing Required Libraries

Explanation:

- TensorFlow → Used to build and train the neural network.
- NumPy → Used for numerical operations and data handling.
- Matplotlib → Used to plot graphs such as loss, accuracy, gradients, and weight updates.

2.Loading and Preprocessing MNIST Dataset :-

1. MNIST dataset is loaded.
2. Each 28×28 image is flattened into a 784-dimensional vector.
3. Pixel values are normalized between 0 and 1.
4. Labels are converted into one-hot encoded vectors (10 output classes).

3. Defining Hyperparameters :-

Input Size → 784 neurons (flattened image).

Hidden Layer → 128 neurons.

Output Layer → 10 neurons (digits 0–9).

Learning Rate → Controls step size of weight update.

Epochs → Number of times full dataset is passed.

Batch Size → Number of samples per mini-batch.

4. Weight and Bias Initialization :-

Weights are initialized randomly.

Biases are initialized as zeros.

`tf.Variable` allows weights to be updated during training.

Initial W1 and W2 values are printed to observe weight learning.

5. Forward Propagation Function:-

   Compute hidden layer linear combination.

   Apply ReLU activation.

   Compute output layer linear combination.

   Apply Softmax activation to get probabilities.

6. Loss Function :-

   Uses Categorical Cross-Entropy loss.

   Measures difference between predicted and actual labels.

   Lower loss means better performance.

7. Accuracy Function:-

   Compares predicted class with actual class.

   Returns percentage of correctly classified samples.

8. Training Loop (Mini-Batch Gradient Descent):-

   Data is shuffled at each epoch.

   Divided into mini-batches.

   Forward propagation is performed.

   Loss is computed.

   Gradients are calculated using GradientTape.

   Weights are updated using gradient descent rule.

9. Tracking Performance:-

The following lists store training behavior:

- loss_history
- accuracy_history
- grad_norm_W1
- grad_norm_W2
- weight_change_W1
- weight_change_W2

These are used to plot graphs:

- Loss vs Epoch
- Accuracy vs Epoch
- Gradient Norm vs Iterations
- Weight Update Magnitude

9. Printing Final Weights :-

```
print("Final W1:", W1.numpy().flatten()[:5])
print("Final W2:", W2.numpy().flatten()[:5])
```

Summary of Performance Evaluation

The three-layer neural network was evaluated using loss, accuracy, gradient norms, and weight update magnitude during training on the MNIST dataset.

At the beginning of training, the model showed:

- High loss (~2.3), indicating random predictions
- Low accuracy (~10%), equivalent to random guessing

As training progressed using mini-batch gradient descent, the following improvements were observed:

- Continuous decrease in cross-entropy loss
- Steady increase in classification accuracy
- Reduction in gradient magnitude over epochs
- Decrease in weight update size near convergence

By the final epoch, the model achieved:

- Low loss (< 0.3)
- High accuracy 97.18%

These results confirm that forward propagation and backpropagation were implemented correctly, and the network successfully learned meaningful features from the MNIST dataset.

Overall, the model demonstrates stable convergence, effective learning, and good classification performance using TensorFlow core operations without Keras.

My Comments :-

1.From this experiment, I learned how a three-layer neural network works internally without using high-level libraries like Keras. I understood how forward propagation calculates outputs using weights, biases, and activation functions.

2.I learned how backpropagation updates the weights by calculating gradients using the chain rule. Using TensorFlow, I was able to see how gradients are computed and how mini-batch gradient descent helps in faster and more stable training.

3.I understood the importance of choosing proper learning rate, initializing weights correctly, normalizing input data

4.By observing the decrease in loss and increase in accuracy after each epoch, I understood how the model learns from loss and improves accuracy.

5.This experiment helped me gain knowledge of neural network implementation, gradient descent, and optimization techniques.

6.Overall, this practical improved my understanding of deep learning concepts and gave me confidence in implementing neural networks from using TensorFlow.